Machete BD: estimación de tamaños y costos

Parcial 2 de Bases de Datos. Son DOS temas distintos que se combinan al final:

- **Tema A, tamaño:** ¿cuántas tuplas y cuántos bloques tiene el resultado de un operador? (usa el factor de selectividad fs)
- **Tema B, costo:** ¿cuántos bloques hay que transferir para calcular un operador? (usa la fórmula de cada algoritmo)

Notación: br, bs = bloques de r y s. nr = tuplas de r. bf = bloques del resultado. hi = altura del índice. V(A, r) = valores distintos de A en r. M = bloques de buffer.

0. Cuentas básicas

Qué	Fórmula
Tamaño de un registro (archivo secuencial)	suma de atributos + 8 (puntero)
Tuplas por bloque	⌊B / tamaño_tupla⌋ (hacia abajo)
Bloques de una tabla	⌈N / tuplas_por_bloque⌉ (hacia arriba)
Claves por nodo B+ (bloque de C bytes)	n = (C − 8) / (TCB + 8)
Altura B+ (K valores distintos de clave)	h ⩽ ⌈log_{⌈n/2⌉}(K)⌉
Cambio de base	log_b(x) = ln(x) / ln(b)

K es la cantidad de valores DISTINTOS de la clave del índice, no los bloques ni los registros totales. El n/2 es la base del logaritmo (un nodo puede estar mitad lleno). Si el enunciado dice 314 claves por nodo podés usar n = 314 o 315; lo habitual es usar lo que dice el enunciado.

TEMA A. Tamaño del resultado

A1. Tuplas del resultado según operador

Operador	Tuplas
Selección	|r| · fs(P, r)

Operador	Tuplas
Reunión (selectiva o natural)	$\|r\| \cdot \|s\| \cdot fs(P, r, s)$
Producto cartesiano	$\|r\| \cdot \|s\|$
Proyección	las mismas que r
Ordenación	las mismas que r
Concatenación r ++ s	$\|r\| + \|s\|$
Intersección	$\leqslant \min(\|r\|, \|s\|)$
Resta	$\leqslant \|r\|$
Eliminar duplicados	$\leqslant \|r\|$

Para pasar a bloques: bloques = ⌈tuplas / tuplas_por_bloque DEL RESULTADO⌉. Si la tupla del resultado es más chica o más grande, cambia cuántas entran por bloque.

A2. fs para SELECCIÓN (supone uniformidad e independencia)

Predicado	fs
A = c	1 / V(A, r)
A ⩾ c	(max − c) / (max − min)
A < c	(c − min) / (max − min + 1)
c ⩽ A < c′	(c′ − c) / (max − min)
P1 y P2 (independencia)	fs(P1) · fs(P2)
no P	1 − fs(P)
P1 o P2	1 − (1 − fs(P1)) · (1 − fs(P2))

Ojo con ">": para edad > 40 la filmina usa c = 41, es decir (100 − 41)/(100 − 1) = 59/99.

Uniformidad: todos los valores del atributo son igual de probables. Independencia: las condiciones sobre atributos distintos no se influyen.

Si el enunciado da directo la fracción (por ejemplo, 1/10 de las ventas son del día x), usá esa fracción como fs.

A3. fs para REUNIÓN (la que no habías visto)

Para la reunión r.A = s.B:

fs = 1 / max(V(A, r), V(B, s))

Por lo tanto: **|r ⋈ s| = |r| · |s| / max(V(A, r), V(B, s))**

De dónde sale: por cada valor común hay |r|/V(A,r) tuplas en r y |s|/V(B,s) en s, y cada par coincide. Hay min(V(A,r), V(B,s)) valores comunes. Multiplicando y simplificando queda la fórmula de arriba.

Casos especiales:

- Si A es clave candidata de r: fs = 1 / max(|r|, V(B, s))
- Si B es clave foránea en s hacia r: fs = 1 / |r|
- Si la reunión es por dos atributos y hay independencia: se multiplican los fs de cada igualdad
- Si las tablas no comparten atributos: es un producto cartesiano, |r| · |s|

Ejemplo: S(b,c) y T(c,d), 1000 tuplas cada una, V(S,c) = 500, V(T,c) = 20. Entonces |S ⋈ T| = 1000 · 1000 / max(500, 20) = 2000.

Ejemplo con clave foránea: estudiante (10.000) e inscripción (40.000), id_estudiante clave foránea. fs = 1/10.000, así que |resultado| = 10.000 · 40.000 / 10.000 = 40.000.

TEMA B. Costo de cada operador físico (en transferencias de bloque)

Se asume el peor caso: un bloque de buffer por tabla.

B1. Selección

Algoritmo	Costo
Recorrido secuencial	br
A = c, índice primario, clave candidata	hi + 1
A = c, índice secundario, clave candidata	hi + 1
A = c, índice primario, NO clave	hi + b (b = bloques con las tuplas que coinciden)

Algoritmo	Costo
A = c, índice secundario, NO clave (buckets)	hi + k + n (k = bloques de bucket, n = registros recuperados)
A ⩾ v, índice primario	hi + b
A ⩽ v, índice primario	no usa índice: escanea desde el principio hasta la primera tupla > v
A ⩾ v, índice secundario	hi + (l − 1) + k + n (l = hojas recorridas)

Idea clave: con índice primario las tuplas que coinciden están contiguas, así que se leen b bloques. Con índice secundario cada registro puede estar en un bloque distinto, de ahí el término n, que puede ser carísimo (peor que recorrer la tabla entera).

Aviso: en una filmina la búsqueda por igualdad con primario no clave figura como hi + 1 + b y en la de reunión como c = hi + b. Confirmá cuál usa tu cátedra.

B2. Proyección

Costo br (lee la tabla). Si se escribe el resultado, se suma bf.

B3. Ordenamiento externo (la tabla no entra en memoria)

1. Generar corridas ordenadas: ⌈br / M⌉ corridas.
2. Mezclar de a M − 1 corridas por pasada.
3. Con k pasadas de mezcla: **br · (2k + 1)** transferencias.

Aviso: una filmina da k = 1 + ⌈log_{M−1}(⌈br/M⌉)⌉, pero la prueba y el ejemplo cuentan solo pasadas de mezcla, que da k = ⌈log_{M−1}(⌈br/M⌉)⌉. Preguntar a la cátedra cuál usar.

B4. Reuniones

Algoritmo	Costo	Cuándo
Loop anidado de bloques	br · bs + br + bf	Siempre se puede. Externa = la tabla chica.
Loop anidado de bloques con M buffers	⌈br / (M − 2)⌉ · bs + br + bf	Hay memoria de sobra.

Algoritmo	Costo	Cuándo
Loop anidado indexado	br + nr · c + bf	Hay índice sobre el atributo de la interna.
Merge join	br + bs + bf (más ordenar si hace falta)	Ambas ordenadas por el atributo.

c = costo de una búsqueda en el índice de s:

Índice	c
Primario, clave candidata	hi + 1
Primario, no clave	hi + b
Secundario, clave candidata	hi + 1
Secundario, no clave	hi + k + n_match

Cuál elegir:

1. Si la interna tiene índice sobre el atributo de reunión: indexado, pero solo conviene si nr · c < br · bs.
2. Si ambas están ordenadas por ese atributo: merge.
3. Si no: loop anidado de bloques con la chica de externa.

Intuición: el de bloques repite poco pero cada vez lee TODA s. El indexado repite una vez por tupla pero cada vez lee solo c bloques. Gana el que tenga menor producto.

B5. Otros operadores

Operador	Costo
Eliminar duplicados	ordenar + bT + bf
Concatenación	br + bs
Resta	ordenar ambas + br + bs + bf
Intersección	ordenar ambas + br + bs + bf

Operador	Costo
Agregación	igual que eliminar duplicados (ordenar y agrupar); sum, min, max, count y avg se calculan al vuelo; avg = sum / count

3. Cómo se combinan: materialización

Cada operador escribe su resultado intermedio a disco y el siguiente lo lee. Las tablas temporales NO tienen índices, así que lo que sale de un operador se lee con recorrido secuencial.

Costo total = suma de los costos de los operadores + suma de los bloques escritos de cada resultado intermedio

El resultado final NO se escribe. Si la fórmula del operador trae un bf, no lo cuentes dos veces: o va dentro del operador o va en la suma de escrituras.

Pasos:

1. Dibujar el árbol (hojas = tablas, nodos = operadores).
2. Calcular los tamaños de las tablas base (en bloques).
3. Para cada nodo, de abajo hacia arriba: calcular fs, tuplas y bloques de su resultado.
4. Elegir el algoritmo de cada nodo y calcular su costo.
5. Sumar costos de operadores y escrituras de intermedios.

Ejemplo de las filminas: proyección de id sobre la selección edad > 40 de persona (20 bloques, 10 tuplas por bloque, edades 1 a 100, índice primario en edad con 20 entradas por nodo).

- fs = 59/99 = 0,595, tuplas = 200 · 0,595 = 119, bloques = 12
- hi = ⌈log_{10}(100)⌉ = 2, selección = hi + b = 14
- escribir el intermedio = 12, proyección lee 12
- **Total = 14 + 12 + 12 = 38**

Ejemplo: π cantidad, valor (σ fecha = x (ventas)), 600 bloques, 1/10 cumple.

- σ sin índice lee 600 y produce 60 bloques (tuplas completas)
- se escriben 60, π lee 60 y su salida (30 bloques) es el resultado final
- **Total = 600 + 60 + 60 = 720**

4. Optimización

Reglas y heurísticas principales:

- Partir selecciones con AND, y las selecciones conmutan.
- La reunión natural es asociativa.
- Empujar selecciones y proyecciones hacia abajo (hacer primero la más restrictiva).
- Reunir primero el par con menor tamaño estimado.
- Ojo: una selección sobre una tabla base produce una tabla temporal SIN índice, y puede hacer perder un algoritmo indexado.

Orden de reuniones con programación dinámica:

- La cantidad de órdenes es $(2(n-1))!/(n-1)!$: 12 para n = 3, 1680 para n = 5.
- Se arma una tabla con un renglón por subconjunto de tablas: tamaño estimado, mejor costo y mejor expresión.
- Tablas base: tamaño y costo 0 (se asumen en memoria). Pares: costo 0.
- Costo de la partición {S1, S2} = mejor costo(S1) + mejor costo(S2) + tamaño(S1) + tamaño(S2), contando 0 el tamaño de una tabla base.
- Se elige la partición de menor costo; la expresión es la reunión de las mejores expresiones de las dos partes.
- Árbol profundo a la izquierda: el lado derecho de cada reunión es siempre una tabla base.

Ejemplo S(b,c), T(c,d), U(d,a), 1000 tuplas cada una, V(S,c) = 500, V(T,c) = 20, V(T,d) = 50, V(U,d) = 1000:

- $|S \bowtie T| = 2000$, $|T \bowtie U| = 1000$, S y U no comparten atributos: $|S \bowtie U| = 10^6$
- Para {S, T, U}: $(T \bowtie U) \bowtie S$ cuesta 1000 (ganadora), $(S \bowtie T) \bowtie U$ cuesta 2000, $(S \bowtie U) \bowtie T$ cuesta 10^6
- $|(S \bowtie T) \bowtie U| = 2000 \cdot 1000 / \max(50, 1000) = 2000$

5. Errores típicos

1. Contar la tupla reducida (después de la proyección) como el tamaño del resultado de la selección: la selección deja tuplas completas.
2. Olvidar que el intermedio se escribe (una vez) y se lee (otra vez).
3. Usar K = total de registros en la altura del B+ en vez de valores distintos de la clave.

4. Poner de externa la tabla grande en el loop anidado.
5. Suponer que el índice siempre conviene: compará nr · c contra br · bs.
6. Sumar bf dos veces.
7. En una resta, intersección o merge, olvidar sumar el costo de ordenar si las tablas no vienen ordenadas.